GRAU EN ENGINYERIA INFORMÀTICA

102753 Visualització Gràfica Interactiva



3.3+.

4. Transformacions

Enric Martí Gòdia

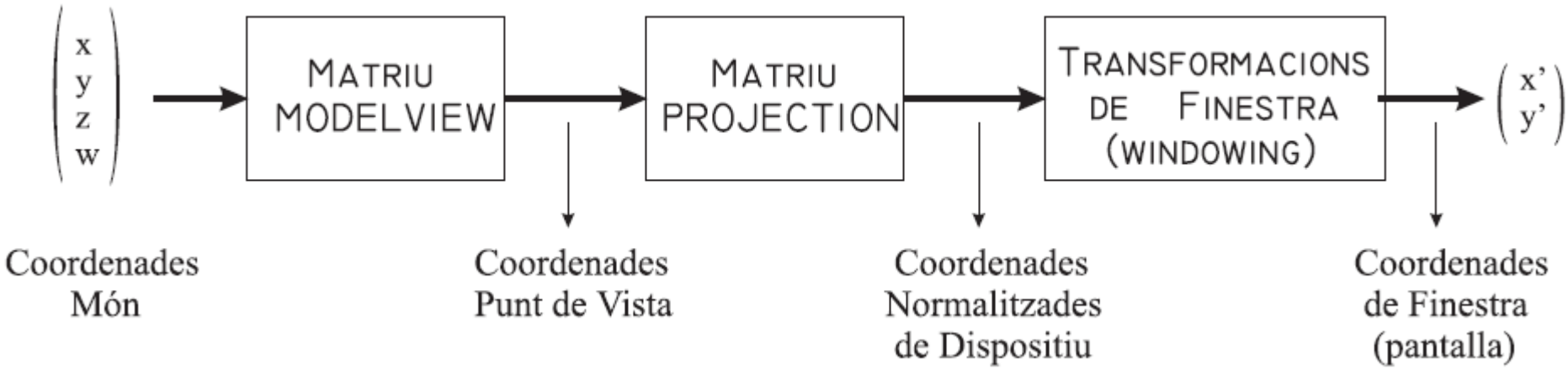
Departament de Ciències de la Computació

Continguts

1. Pipeline de Visualització
2. Vectors i Matrius en GLM
3. Transformacions Geomètriques en GLM
4. Transformacions de Visualització en GLM
5. Pas de Matrius GLM als shaders
6. Exemples

1. Pipeline de Visualització

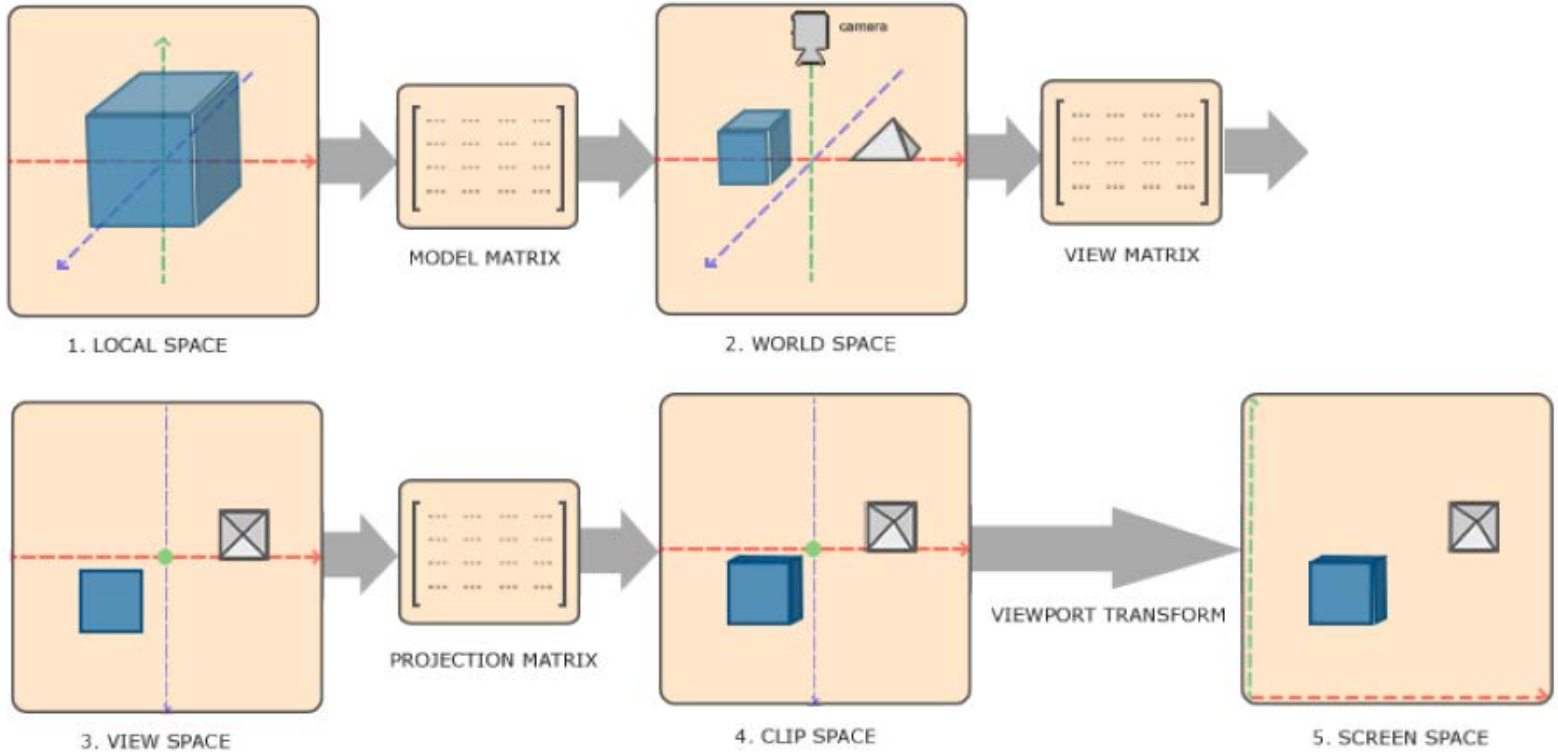
Pipeline de Visualització GL 1.1:



Stack de matrius (`GL_MODELVIEW, GL_PROJECTION`) i comandes de transformacions (`glTranslatef(), glRotatef(), glOrtho(), gluLookAt()`) de GL 1.1 deprecades en GL 3.3+.

1. Pipeline de Visualització

Pipeline de Visualització GL 3.3+:



GL 3.3+ no té comandes de manipulació de matrius ni definició explícita de comandes de transformacions geomètriques (TG) o de visualització (TV).

1. Pipeline de Visualització

Pipeline de Visualització GL 3.3+:

- Transformació de Viewport ja implementada en l'aplicació amb la comanda GL `glViewport()`.
- Cal implementar en l'aplicació i en shaders (pipeline) les tres primeres matrius (`ModelMatrix`, `ViewMatrix`, `ProjectionMatrix`). Aconsellable utilitzar alguna llibreria externa com GLM (*OpenGL Mathematics*), passant les matrius calculades als shaders pel càlcul del pipeline.

1. Pipeline de Visualització

GLM (*OpenGL Mathematics*):

- http://glm.g-truc.net/0.9.5/index.html
- Llibreria header-only, sense fitxers .lib o .dll

```
#include <glm/glm.hpp>// Include all GLM core/GLSL features
#include <glm/ext>    // Include all GLM extensions
using namespace glm;
```

- Llibreria compatible en tipus (`vec*, mat*`) i estructura i accés de vectors (swizzling) amb el llenguatge GLSL.
- Comandes de TG i de TV amb sintaxi similar a les comandes GL 1.1, amb la diferència que les comandes GLM treuen de sortida la matriu que cal gestionar explícitament en l'aplicació.

2. Vectors i Matrius en GLM

GLM gestiona les transformacions (geomètriques i de visualització) mitjançant matrius de 4x4 de la forma

```
glm::vec4 vec(1.0f, 0.0f, 0.0f, 1.0f); // Vector 4 floats.
glm::mat4 trans(1.0); // Matriu Identitat 4x4.
trans = glm::translate(trans, glm::vec3(1.0f, 1.0f, 0.0f));
vecT = trans * vec;
```

on `vec=(x,y,z,1.0)` correspon a un punt 3D en coordenades homogènies, `trans` la matriu de transformació i `vecT=(x',y',z',1.0)` el punt 3D transformat.

GLM incorpora un gran nombre de funcions matemàtiques i de tractament de vectors i matrius que implementa GLSL. També opera amb quaternions.

2. Vectors i Matrius en GLM

Comandes de gestió de vectors:

- Definició i inicialització de vectors:
  - `vecn` (n columnes `float`): `vec4, vec3, vec2`
  - `dvecn` (n columnes `double`): `dvec4, dvec3, dvec2`
  - Inicialització:

```
glm::vec4 V(0.0f,5.0f,30.0f,1.0f);
glm::vec4 V(1.0f); ←→ glm::vec4 V(1.0f,1.0f,1.0f,1.0f);
```

- Operadors aritmètics de vectors (+,-,*,/) component a component.

```
glm::vec4 v1(0.0f,1.0f,2.0f,1.0f);
glm::vec4 v2(3.0f,4.0f,5.0f,1.0f);
glm::vec4 v3 = v1 + v2;    // v3 = (3.0,5.0f,7.0f,2.0f);
glm::vec4 v3 = v2 – v1;    // v3 = (3.0,3.0f,3.0f,0.0f);
glm::vec4 v3 = v1 * v2;    // v3 = (0.0,4.0f,10.0f,1.0f);
```

2. Vectors i Matrius en GLM

Comandes de gestió de matrius:

- Definició i inicialització de matrius:
  - `matnxm,matn` (n columnes, `m` files `float`): `mat4x3, mat4,` etc.
  - `dmatnxm,dmatn` (n columnes, `m` files `double`): `dmat4x3, dmat4,`etc.
  - Inicialització matriu a identitat: `glm::mat4 View(1.0);`
  - Matriu identitat: `glm::identity();`
- Multiplicació de matrius: Operador *, especificant ordre.

Comandes de gestió de matrius:

```
glm::mat4 computeModelMatrix(float Translate, glm::vec2 const &Rotate) {
 glm::mat4 View=glm::translate(glm::identity(),glm::vec3(0.0f,0.0f,-Translate));
 View = glm::rotate(View, Rotate.y, glm::vec3(0.0f, 1.0f, 0.0f));
 View = glm::rotate(View, Rotate.x, glm::vec3(1.0f, 0.0f, 0.0f));
 glm::mat4 Model = glm::scale(glm::identity(), glm::vec3(0.5f));
 return View * Model;
}
```

2. Vectors i Matrius en GLM

Comandes de gestió de matrius (2):

- Ordenació files i columnes com a C. Ordre per columnes:

$$\begin{pmatrix} m_1 & m_5 & m_9 & m_{13} \\ m_2 & m_6 & m_{10} & m_{14} \\ m_3 & m_7 & m_{11} & m_{15} \\ m_4 & m_8 & m_{12} & m_{16} \end{pmatrix}$$

- `glm::mat4 View(1.0);`

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2. Vectors i Matrius en GLM

Comandes de gestió de matrius (3):

- Introduir valors a matrius per a transformacions no pre-definides:

```
glm::mat4 View(1.0);
View[0] = glm::vec4(a,b,c,d);
View[1] = glm::vec4(e,f,g,h);
View[2] = glm::vec4(i,j,k,l);
View[3] = glm::vec4(m,n,o,p);
```

$$\begin{pmatrix} a & e & i & m \\ b & f & j & n \\ c & g & k & o \\ d & h & l & p \end{pmatrix}$$

2. Vectors i Matrius en GLM

Comandes de gestió de matrius (4):

- Apuntador a memòria d'una variable GLM `glm::value_ptr()` :

```
glm::vec4 v(0.0f);
glm::mat4 m(1.0f);
…
glVertex3fv(glm::value_ptr(v));
glLoadMatrixfv(glm::value_ptr(m));
```

Que equival a:

```
glm::vec4 v(0.0f);
glm::mat4 m(1.0f);
…
glVertex3fv(&v[0]);
glLoadMatrixfv(&m[0][0]);
```

3. Transformacions Geomètriques en GLM

GLM defineix comandes explícites per a les Transformacions Geomètriques de GL 1.1 amb el format:

```
glm::mat4 funcioTG(glm::mat4 M_in, glm::vec3(parx,pary,parz));
```

que realitzen tres accions:

1) Crear la matriu de transformació `M_tg` a partir dels paràmetres `glm::vec3(parx,pary,parz)` donats en la comanda `funcioTG`.

2) Multiplicar la matriu `M_in` per `M_tg` per la dreta

```
M_res = M_in * M_tg
```

3) Retornar `M_res`

IMPORTANT: Multiplicació de matrius per la dreta.

3. Transformacions Geomètriques en GLM

Multiplicació de matrius: Degut a que les matrius es multipliquen per la dreta, **cal definir en el codi font les transformacions EN ORDRE INVERS a com s'aplicaran als vèrtex**.

Exemple: Volem aplicar tres transformacions M_S, M_R, M_T al vèrtex v de la forma $M_T\,[M_R\,(\,M_S\,v)\,]$

En el codi font s'especificaria de la següent forma:

```
glm::mat4 M(1.0f);                                                    // M ← I
M = glm::translate(glm::identity(),glm::vec3(1.0f,2.0f,10.0f);// M ←I*MT=MT
M = glm::rotate(M,radians(45.0f),glm::vec3(1.0f,0.0f,0.0f);// M ←M*MR=MT*MR
M = glm::scale(M,glm::vec3(1.0f,0.0f,0.0f);    // M ← M * MS = MT * MR * MS
<Activació shader>
<Pas de matriu M al shader>
<dibuix del vèrtex v de l'escena>          // Càlcul v' = M * v al shader
```

3. Transformacions Geomètriques en GLM

Comandes de Transformacions Geomètriques:

- Traslació → `glm::translate()`
- Rotació → `glm::rotate()`
- Escalat → `glm::scale()`
- Per a qualsevol altra transformació (per exemple *shear*), inicialitzar la matriu `glm::mat4` i acumular-la (multiplicar) a la que construïm.

3. Transformacions Geomètriques en GLM

```
glm::mat4 glm::translate(glm::mat4 const& M, glm::vec3 const& translation);

glm::dmat4 glm::translate(glm::dmat4 const& M, glm::vec3 const& translation);
```

Acció: Funció que defineix la matriu de translació `Mt` corresponent al vector de translació `translation=(Tx,Ty,Tz)`, la multiplica per la dreta per la matriu `M` i retorna el resultat de la multiplicació.

`glm::translate()` ← `M Mt`

$$M_t = \begin{pmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

3. Transformacions Geomètriques en GLM

```
glm::mat4 glm::translate(glm::mat4 const& M, glm::vec3 const& translation);

glm::dmat4 glm::translate(glm::dmat4 const& M, glm::vec3 const& translation);
```

Exemple:

```
glm::mat4 M(1.0f);                                    // M ← I
M = glm::translate(M,glm::vec3(1.0f,2.0f,10.0f); // M ← M * MT = I * MT = MT
M = glm::translate(M,glm::vec3(1.0f,2.0f,10.0f); // M ← M * MT2 = MT * MT2

<Activació shader>
<Pas de matriu M al shader>
Dibuixa_Escena();        // Dibuix de l'escena al buffer OpenGL,
                         // Càlcul v' = M * v al shader (v vèrtex escena)

swapBuffers();           // Transferir buffer OpenGL a buffer pantalla.
```

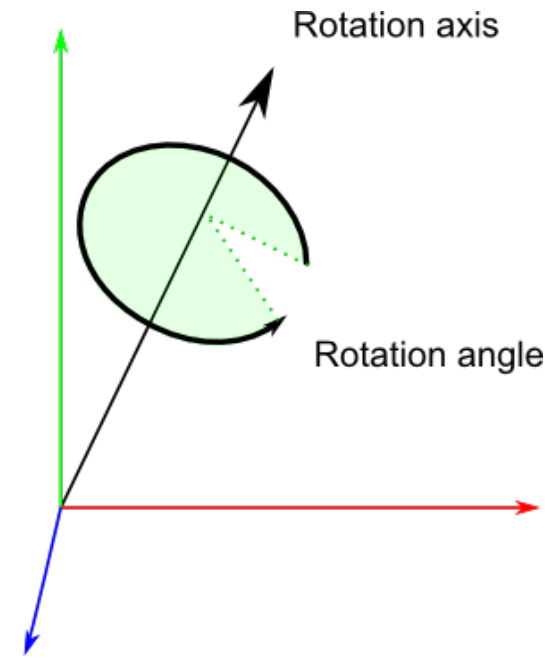
3. Transformacions Geomètriques en GLM

```
glm::mat4 glm::rotate(glm::mat4 const& M,float angle,glm::vec3 const& axis);

glm::dmat4 glm::rotate(glm::dmat4 const& M,float angle,glm::vec3 const& axis);
```

Acció: Funció que defineix la matriu de rotació `Ms` corresponent a l'angle angle i eix de rotació `axis=(Ex,Ey,Ez)`, la multiplica per la dreta per la matriu `M` i retorna el resultat de la multiplicació.

`glm::rotate()` ← `M Mr`



3. Transformacions Geomètriques en GLM

```
glm::mat4 glm::rotate(glm::mat4 const& M,float angle,glm::vec3 const& axis);

glm::dmat4 glm::rotate(glm::dmat4 const& M,float angle,glm::vec3 const& axis);
```

Comanda `rotate()` molt potent doncs permet definir un eix de rotació arbitrari o els eixos de coordenades:

`rotate(M,α,glm::vect3(0.0,0.0,1.0))` → Eix Z

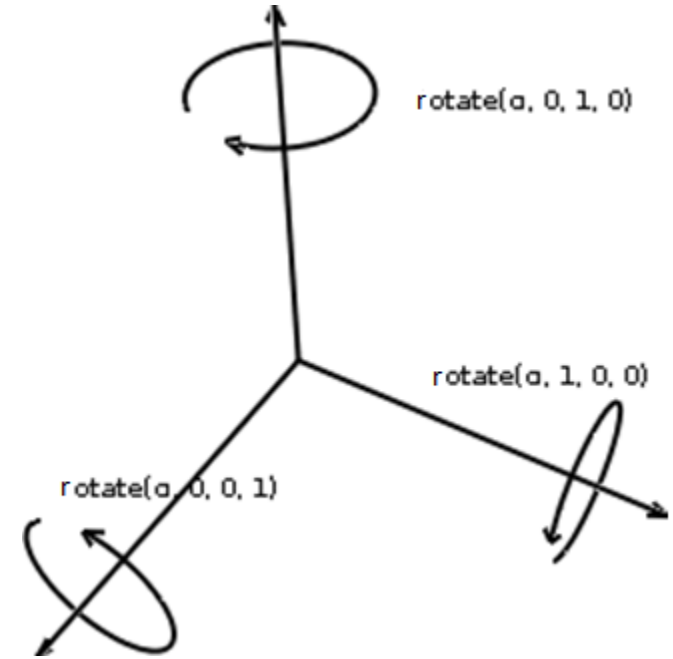
$$M_r = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 & 0 \\ \sin\alpha & \cos\alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

`rotate(M,α,glm::vec3(1.0,0.0,0.0))` → Eix X

$$M_r = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha & 0 \\ 0 & \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

`rotate(M,α,glm::vec3(0.0,1.0,0.0))` →Eix Y

$$M_r = \begin{pmatrix} \cos\alpha & 0 & \sin\alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\alpha & 0 & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



3. Transformacions Geomètriques en GLM

```
glm::mat4 glm::rotate(glm::mat4 const& M,float angle,glm::vec3 const& axis);

glm::dmat4 glm::rotate(glm::dmat4 const& M,float angle,glm::vec3 const& axis);
```

Exemple:

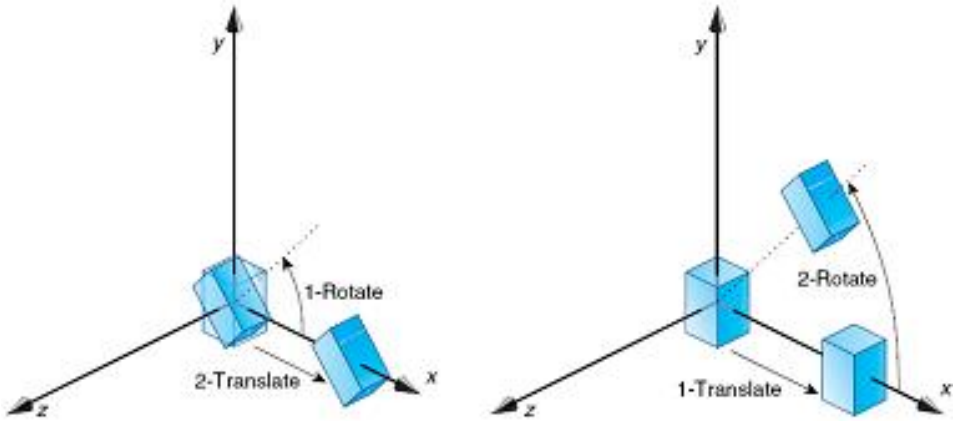
```
glm::mat4 M(1.0f);                                          // M ← I
M = glm::rotate(M,radians(45.0f),glm::vec3(1.0f,0.0f,0.0f);
                                                            // MR ← Rot. eix X 45 gr.
                                                            // M ← M * MR = I * MR = MR
M = glm::rotate(M,radians(60.0f),glm::vec3(0.0f,0.0f,1.0f);
                                                            // MR2 ← Rot. eix Z 60 gr.
                                                            // M ← M * MR2 = MR * MR2

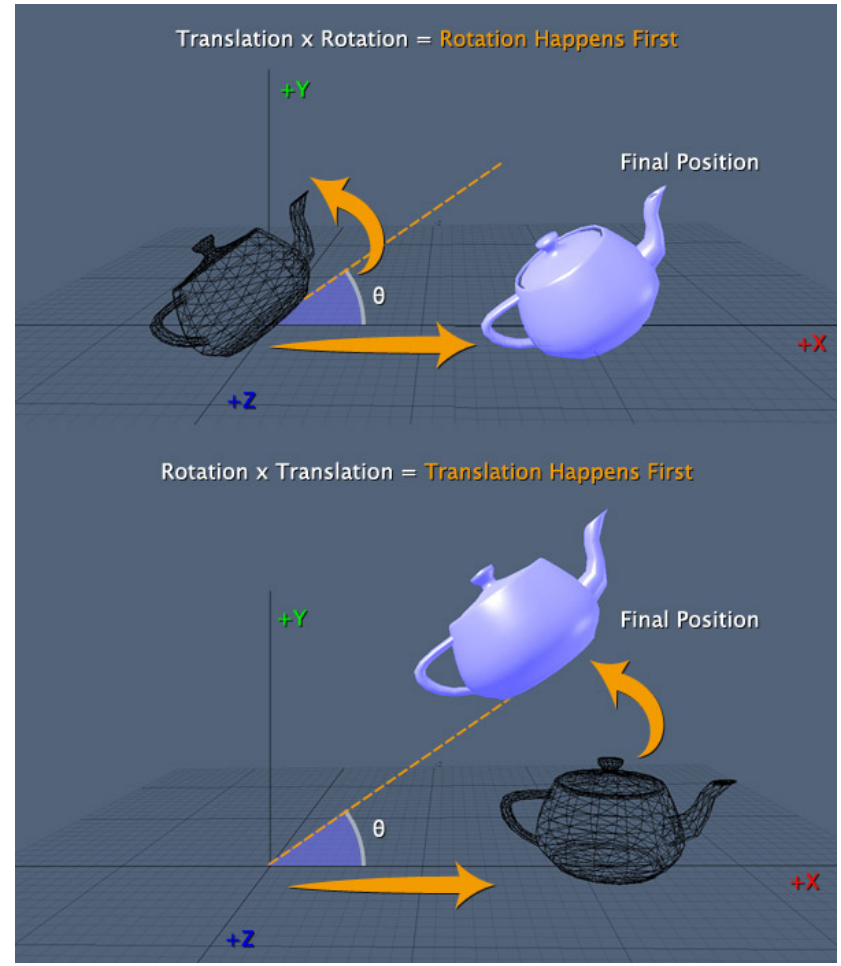

<Activació shader>
<Pas de matriu M al shader>
Dibuixa_Escena();          // Dibuix de l'escena al buffer OpenGL,
                           // Càlcul v' = M * v al shader (v vèrtex escena)

swapBuffers();             // Transferir buffer OpenGL a buffer pantalla.
```

3. Transformacions Geomètriques en GLM

Recordeu: Transformacions Geomètriques **NO** són commutatives.





3. Transformacions Geomètriques en GLM

```
glm::mat4 glm::scale(glm::mat4 const& M, glm::vec3 const& factors);

glm::dmat4 glm::scale(glm::dmat4 const& M, glm::vec3 const& factors);
```

Acció: Funció que defineix la matriu d'escalat `Ms` corresponent al vector d'escalat `factors=(Sx,Sy,Sz)`, la multiplica per la dreta per la matriu `M` i retorna el resultat de la multiplicació.

`glm::scale()` ← `M Ms`

$$M_s = \begin{pmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

3. Transformacions Geomètriques en GLM

```
glm::mat4 glm::scale(glm::mat4 const& M, glm::vec3 const& factors);

glm::dmat4 glm::scale(glm::dmat4 const& M, glm::vec3 const& factors);
```

Exemple:

```
glm::mat4 M(1.0f);                                  // M ← I
M = glm::scale(M,glm::vec3(1.0f,2.0f,5.0f);         // M ← M * MS = I * MS = MS
M = glm::scale(M,glm::vec3(2.0f,2.0f,3.0f);         // M ← M * MS2 = MS * MS2
<Activació shader>
<Pas de matriu M al shader>
Dibuixa_Escena();          // Dibuix de l'escena al buffer OpenGL,
                           // Càlcul v' = M * v al shader (v vèrtex escena)

swapBuffers();             // Transferir buffer OpenGL a buffer pantalla.
```

3. Transformacions Geomètriques en GLM

Transformació arbitrària: Esllavissament respecte eix Z

Matriu:

$$(x'_i, y'_i, z'_i, 1) = \begin{pmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_i \\ y_i \\ z_i \\ 1 \end{pmatrix}$$

Implementació:

```
glm::mat4 M(1.0f);
…
glm::mat4 Msh(1.0f);                    // Msh ← I
Msh[0] = vec4(1.0f,0.0f,0.0f,0.0f);     // Primera columna
Msh[1] = vec4(0.0f,1.0f,0.0f,0.0f);     // Segona columna
Msh[2] = vec4(a,b,1.0f,0.0f);           // Tercera columna
Msh[3] = vec4(0.0f,0.0f,0.0f,1.0f);     // Quarta columna
M = M * Msh;
<Activació shader>
<Pas de matriu M al shader>
Dibuixa_Escena();  // Dibuix de l'escena al buffer OpenGL,
                           // Càlcul v' = M * v al shader (v vèrtex escena)
swapBuffers();             // Transferir buffer OpenGL a buffer pantalla.
```

4. Transformacions de Visualització en GLM

Transformen un vèrtex en Coordenades Món a Coordenades Pantalla.

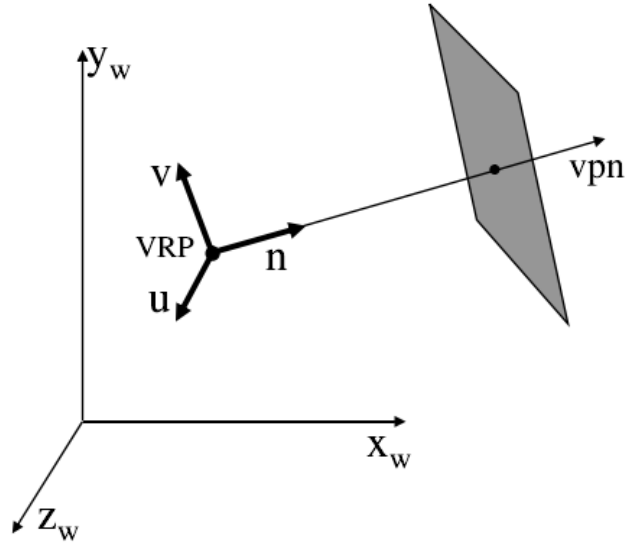
Totes les transformacions són representables en matrius 4x4.

Passos (en ordre invers en com es criden en codi font):

1. Càmera → `glm::lookAt()`
2. Projecció → `glm::ortho()`, `glm::furstrum()`, `glm::perspective()`
3. Viewport → `glViewport()`

4.1. Càmera

Comanda per a situar la càmera definint la posició (VRP) i els eixos $\vec{v}, \vec{n}$ (eix $\vec{u}$ es calcula tenint en compte un sistema de coordenades a esquerres).



La comanda defineix la matriu de canvi de base a partir dels paràmetres.

Comanda `glm::lookAt()`.

4.1. Càmera

```
glm::mat4 glm::lookAt(glm::vec3 const& eye, glm::vec3 const& center, glm::vec3 const& up);
```

Acció: Funció que retorna la matriu de canvi de base `Mc` corresponent a la posició i eixos de càmera definits en els paràmetres.

`glm::lookAt` ← `Mc`

Paràmetres:

`eye=(eyex,eyey,eyez)`: Posició de la càmera en coordenades món.

`center=(centerx,centery,centerz)`: Punt que volem que miri la càmera. Vector $\vec{n}$ es calcula com `(nx,ny,nz)=(centerx-eyex,centery-eyey,centerz-eyez)`

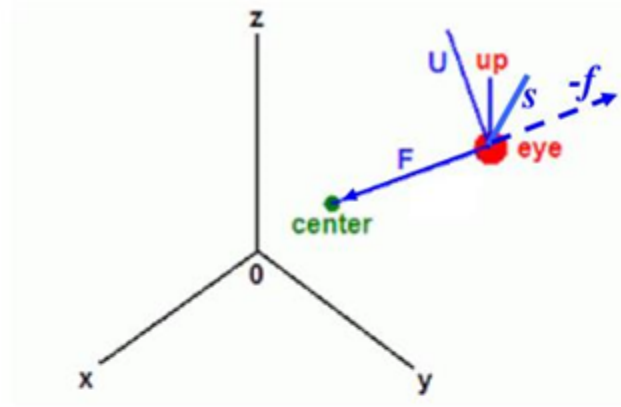
`up=(upx,upy,upz)`: Eix $\vec{v}$ de la càmera.

4.1. Càmera

Details of glm::lookAt (eye, center, up)

$$F = \begin{pmatrix} \text{centerX} - \text{eyeX} \\ \text{centerY} - \text{eyeY} \\ \text{centerZ} - \text{eyeZ} \end{pmatrix}$$

$$f = \frac{F}{\|F\|} \qquad s = f \times \text{UP} \qquad s = \frac{s}{\|s\|} \qquad u = s \times f$$



Change of Basis (after the translation to *eye*)

$$\begin{bmatrix} world_x \\ world_y \\ world_z \end{bmatrix} = \underbrace{\begin{bmatrix} s[0] & u[0] & -f[0] \\ s[1] & u[1] & -f[1] \\ s[2] & u[2] & -f[2] \end{bmatrix}}_{L} \begin{bmatrix} eye_x \\ eye_y \\ eye_z \end{bmatrix}$$

$$x_{world} = L x_{eye} \Rightarrow x_{eye} = L^{-1} x_{world} = L^T x_{world}$$

$$M = \begin{pmatrix} \overbrace{\begin{bmatrix} s[0] & s[1] & s[2] \\ u[0] & u[1] & u[2] \\ -f[0] & -f[1] & -f[2] \end{bmatrix}}^{L^T} & \begin{matrix} 0 \\ 0 \\ 0 \end{matrix} \\ \begin{matrix} 0 & 0 & 0 \end{matrix} & 1 \end{pmatrix}$$

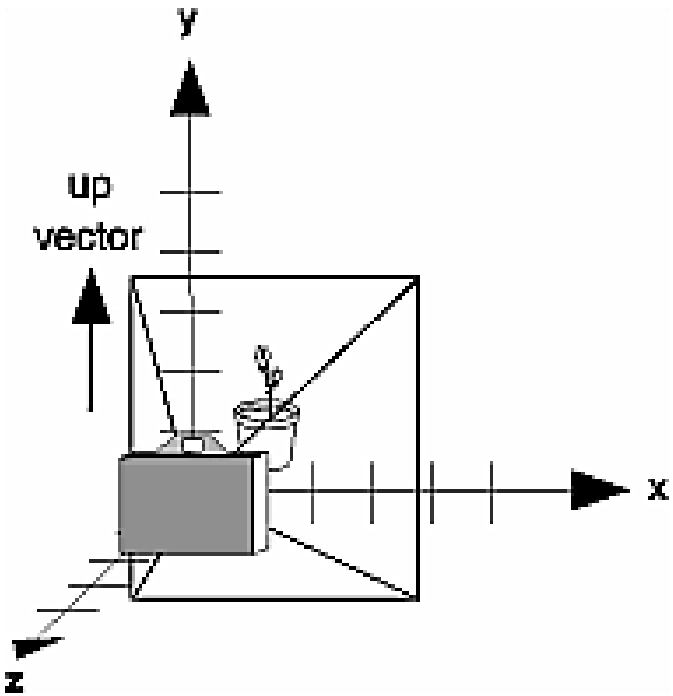
Equivalent to:

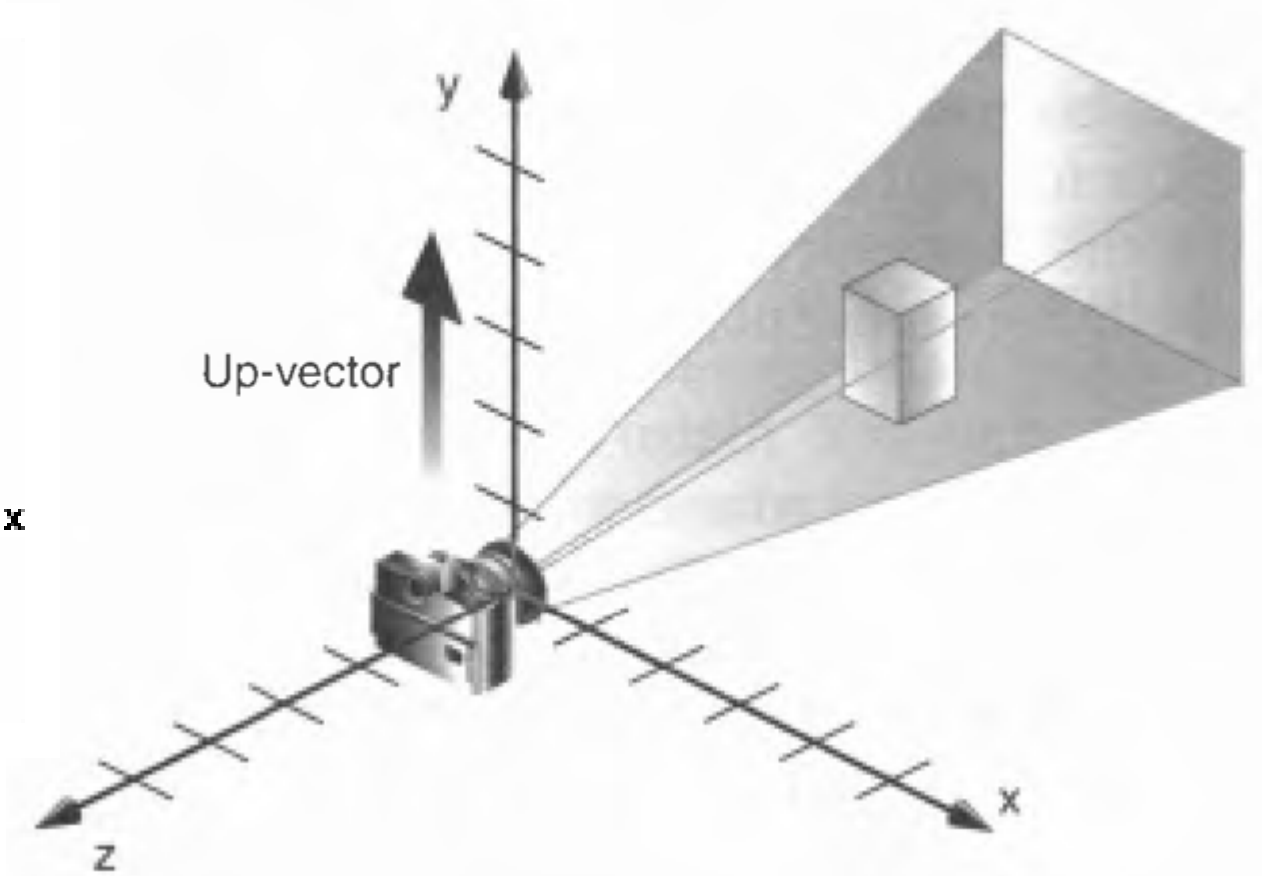
```
M'=glm::translate(M,glm::vec3(-eyez,-eyey,-eyez))
```

4.1. Càmera

Exemple de càmera:

```
glm::mat4 ViewMatrix = glm::lookAt(glm::vec3(0.0f,0.0f,0.0f),glm::vec3(0.0f,0.0f,-1.0f),
glm::vec3(0.0f,1.0f,0.0f);
```





4.1. Càmera

Exemple:

```
glViewport(0,0,w,h);  // Definir Viewport de la Vista (tota la pantalla)

// Definició projecció Ortogràfica | Perspectiva
glm:mat4 ProjectionMatrix = glm::ortho(-10.0f,10.0f,-10.0f,10.0f,0.1f,200.0f);
|
glm::mat4      ProjectionMatrix      =      glm::perspective(radians(60.0f),
1.0f,0.1f,200.0f);
glm::mat4      ViewMatrix      =      glm:lookAt(glm::vec3(10.0f,0.0f,0.0f),
glm::vec3(0.0f,0.0f,0.0f),glm::vec3(0.0f,0.0f,1.0f)); // Definició càmera.

<Activació shader>
<Pas de matrius ProjectionMatrix, ViewMatrix al shader>
Dibuixa_Escena();        // Dibuix de l'escena al buffer OpenGL,
                         // incloent TG als objectes.
swapBuffers();           // Transferir buffer OpenGL a buffer pantalla.
```

4.2. Projecció

Transformació que redueix la dimensionalitat de l'espai (3D→2D).

GLM té funcions per a la projecció ortogràfica i perspectiva.

Qualsevol projecció (cabellera, cabinet, etc.) que es pugui definir en una matriu 4x4 es pot definir explícitament en una matriu `glm::mat4`, posar els valors i multiplicar-la per la resta de matrius del pipeline.

4.2. Projecció

Comandes de Transformacions Geomètriques:

- Projecció Ortogràfica → `glm::ortho()`
- Projecció Perspectiva → `glm::frustum()`
- Projecció Perspectiva amb Volum de Visualització centrat al centre de projecció (punt de càmera) → `glm::perspective()`

4.2. Projecció

```
glm::mat4 glm::ortho(float left, float right, float bottom, float top, float znear, float zfar);
```

```
glm::dmat4 glm::ortho(double left, double right, double bottom, double top, double znear, double zfar);
```

Acció: Funció que retorna la matriu de projecció ortogràfica `Mo`. Prèviament aplica el retall de l'escena segons el Volum de Visualització definit en els paràmetres(`left, right, bottom, top, znear, zfar`).

`glm::ortho` ← `Mo`

Paràmetres:

(`left,right`): Mínim i màxim per l'eix u (x).

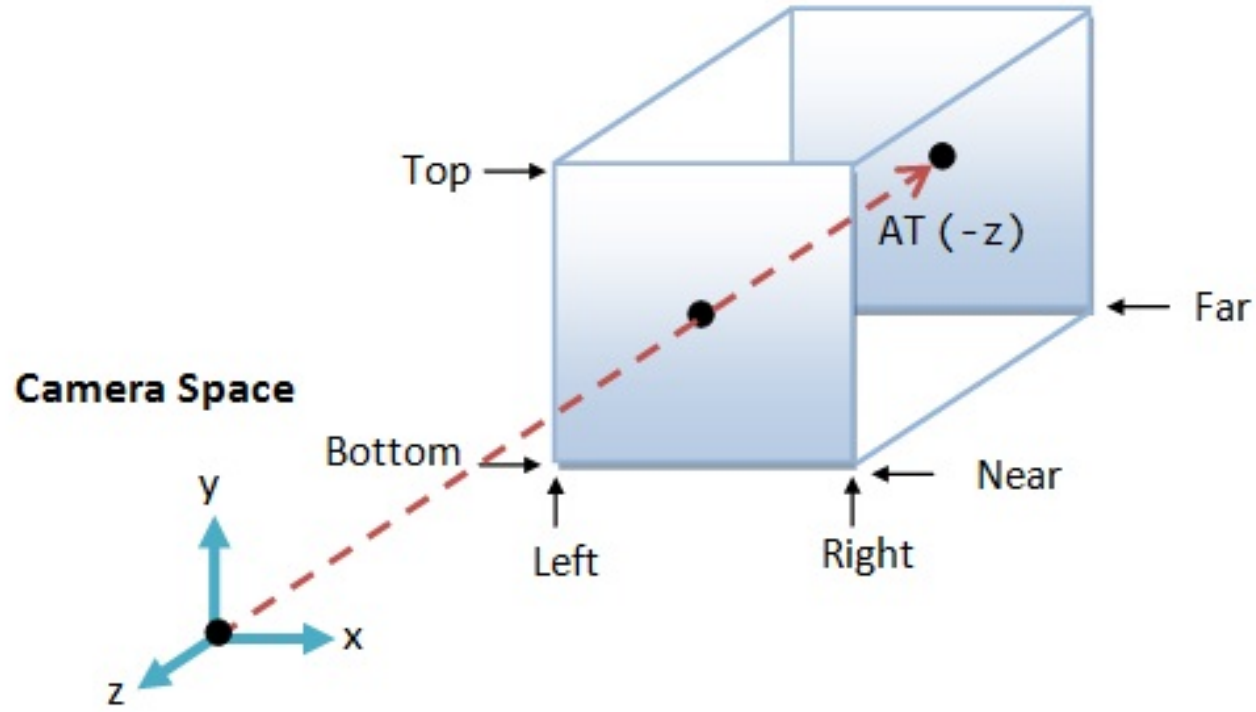
(`bottom,top`): Mínim i màxim per l'eix v (y).

(`near,far`): Mínim i màxim per l'eix n (z).

4.2. Projecció

```
glm::mat4 glm::ortho(float left, float right, float bottom, float top, float znear, float zfar);

glm::dmat4 glm::ortho(double left, double right, double bottom, double top, double znear, double zfar);
```

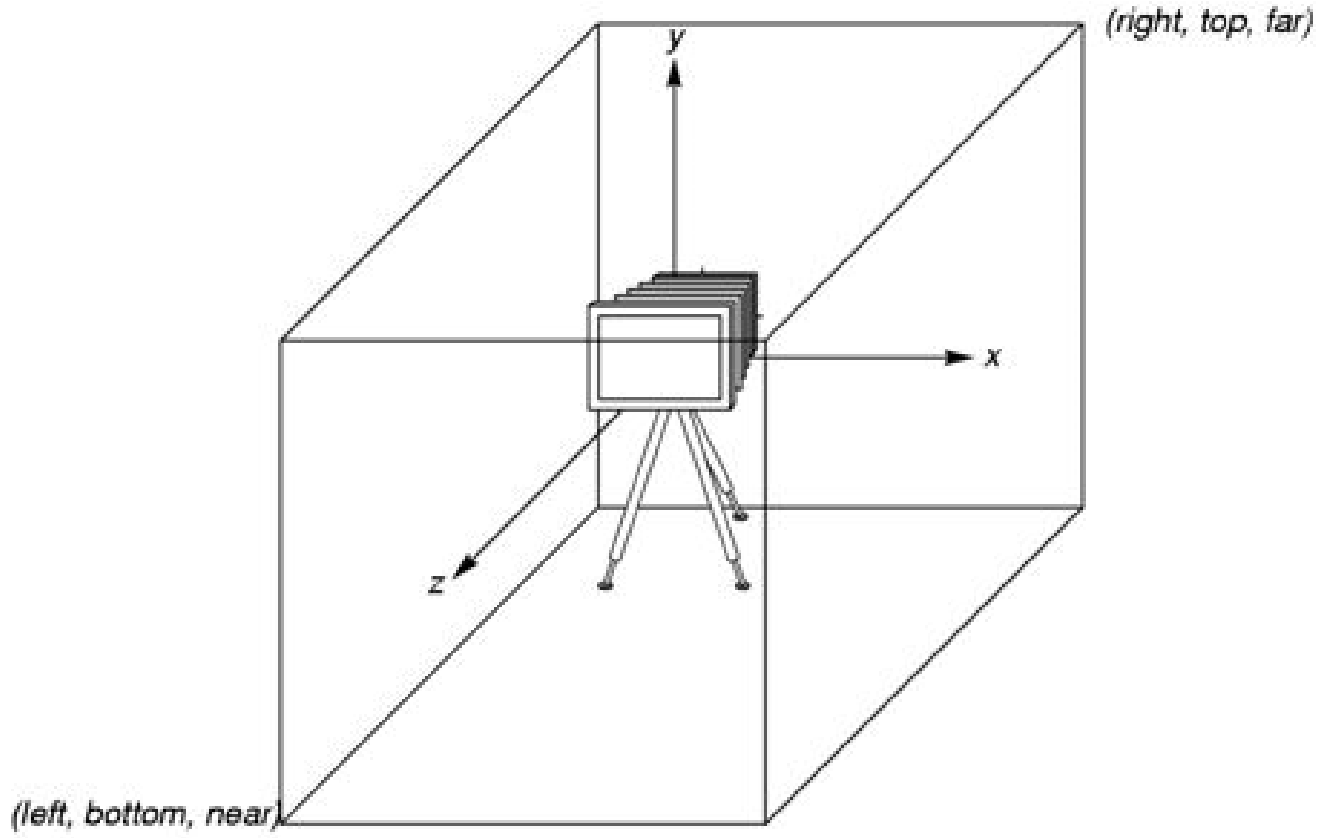


Orthographic Projection: Camera positioned infinitely far away at $z = \infty$

4.2. Projecció

Exemple de projecció ortogràfica:

```
glm::ortho(-1.0f,1.0f, -1.0f,1.0f, -1.0f,1.0f);
```



4.2. Projecció

```
glm::mat4 glm::ortho(float left, float right, float bottom, float top,
float znear, float zfar);

glm::dmat4 glm::ortho(double left, double right, double bottom, double
top, double znear, double zfar);
```

Exemple:

```
glViewport(0,0,w,h);     // Definir Viewport de la Vista (tota la pantalla)

// Definició projecció Ortogràfica
glm:mat4 ProjectionMatrix = glm::ortho(-10.0f,10.0f,-10.0f,10.0f,0.1f,200.0f);
glm::mat4       ViewMatrix       =       glm:lookAt(glm::vec3(10.0f,0.0f,0.0f),
glm::vec3(0.0f,0.0f,0.0f),glm::vec3(0.0f,0.0f,1.0f)); // Definició càmera.

<Activació shader>
<Pas de matrius ProjectionMatrix, ViewMatrix al shader>
Dibuixa_Escena();        // Dibuix de l'escena al buffer OpenGL,
                         //    incloent TG als objectes.
swapBuffers();           // Transferir buffer OpenGL a buffer pantalla.
```

4.2. Projecció

```
glm::mat4 glm::frustrum(float left, float right, float bottom,
float top, float znear, float zfar);

glm::dmat4 glm::frustrum(double left, double right, double bottom,
double top, double znear, double zfar);
```

Acció: Funció que retorna la matriu de projecció perspectiva `Mf`. Prèviament aplica el retall de l'escena segons el Volum de Visualització definit en els paràmetres(`left, right, bottom, top, znear, zfar`).

`glm::frustrum()` ← `Mf`

Paràmetres:

(`left,right`): Mínim i màxim per l'eix u (x).

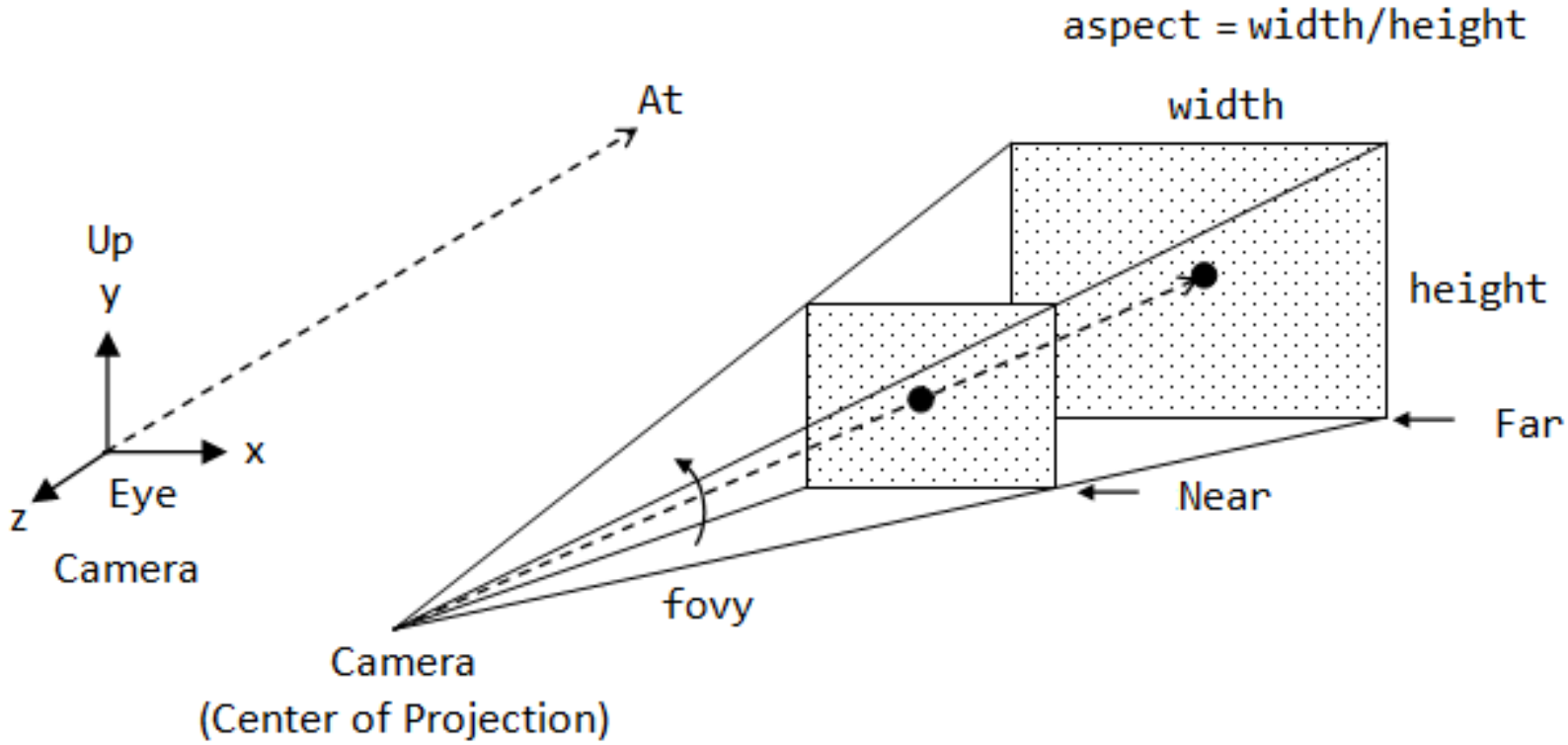
(`bottom,top`): Mínim i màxim per l'eix v (y).

(`znear,zfar`): Mínim i màxim per l'eix n (z).

4.2. Projecció

```
glm::mat4 glm::frustrum(float left, float right, float bottom,
float top, float znear, float zfar);

glm::dmat4 glm::frustrum(double left, double right, double bottom,
double top, double znear, double zfar);
```



4.2. Projecció

```
glm::mat4 glm::frustrum(float left, float right, float bottom, float top, float znear, float zfar);

glm::dmat4 glm::frustrum(double left, double right, double bottom, double top, double znear, double zfar);
```

Exemple:

```
glViewport(0,0,w,h);     // Definir Viewport de la Vista (tota la pantalla)

// Definició projecció Ortogràfica
glm:mat4 ProjectionMatrix = glm::frustrum(-10.0f,10.0f,-10.0f,10.0f,0.1f,200.0f);
glm::mat4      ViewMatrix      =      glm:lookAt(glm::vec3(10.0f,0.0f,0.0f),
glm::vec3(0.0f,0.0f,0.0f),glm::vec3(0.0f,0.0f,1.0f)); // Definició càmera.

<Activació shader>
<Pas de matrius ProjectionMatrix, ViewMatrix al shader>
Dibuixa_Escena();        // Dibuix de l'escena al buffer OpenGL,
                         //    incloent TG als objectes.
swapBuffers();           // Transferir buffer OpenGL a buffer pantalla.
```

4.2. Projecció

```
glm::mat4 glm::perspective(float fovy, float aspect, float znear, float zfar);
```

```
glm::mat4 glm::perspective(double fovy, double aspect, double znear, double zfar);
```

Acció: Funció que retorna la matriu de projecció perspectiva `Mp`. Prèviament aplica el retall de l'escena segons el Volum de Visualització definit en els paràmetres(`fovy, aspect, znear, zfar`).

`glm::perspective()` ← `Mp`

Paràmetres:

(`fovy`): Angle del camp de visió en radians entre (0,π).

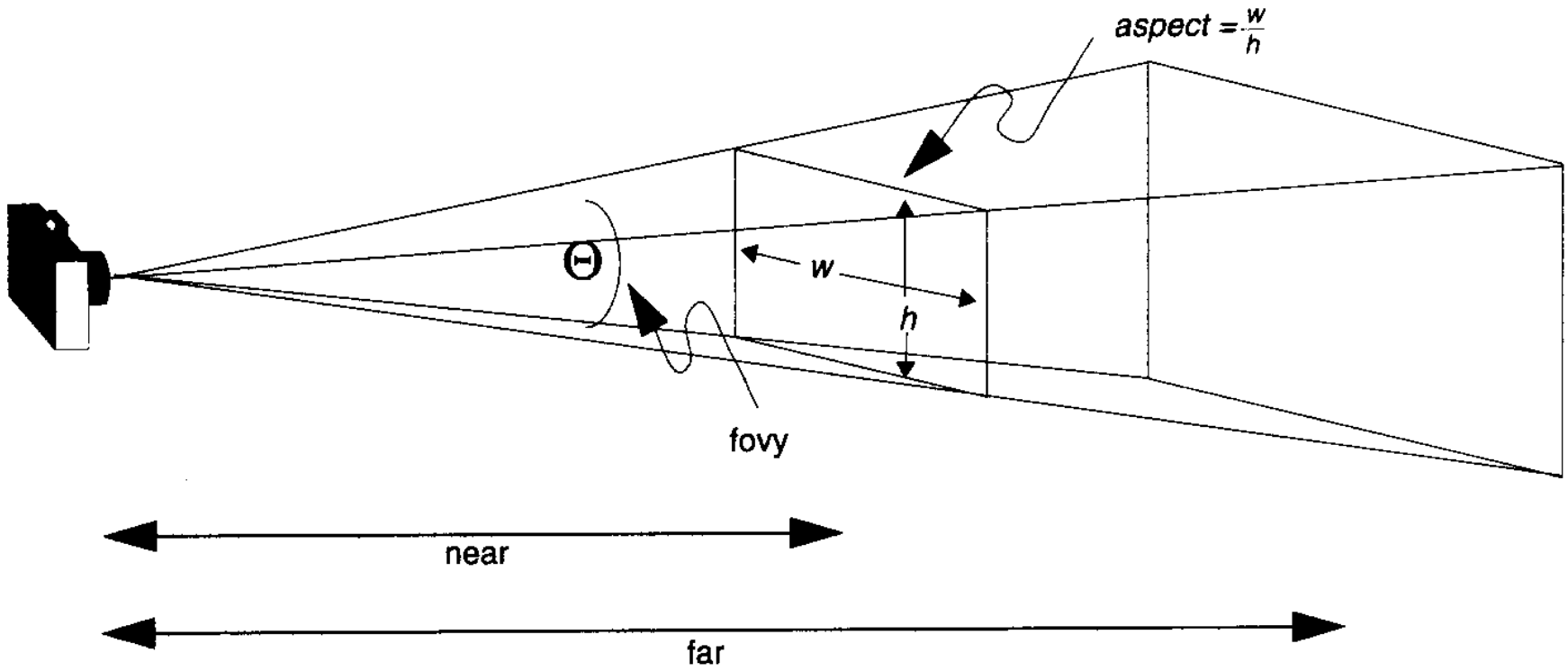
(`aspect`): Relació d'aspecte entre amplada (w) i alçada (h) → $aspect = \frac{w}{h}$

(`znear,zfar`): Mínim i màxim per l'eix n (z).

4.2. Projecció

```
glm::mat4 glm::perspective(float fovy, float aspect, float znear, float zfar);

glm::mat4 glm::perspective(double fovy, double aspect, double znear, double zfar);
```



4.2. Projecció

```
glm::mat4 glm::perspective(float fovy, float aspect, float znear, float zfar);

glm::mat4 glm::perspective(double fovy, double aspect, double znear, double zfar);
```

Exemple:

```
glViewport(0,0,w,h);     // Definir Viewport de la Vista (tota la pantalla)

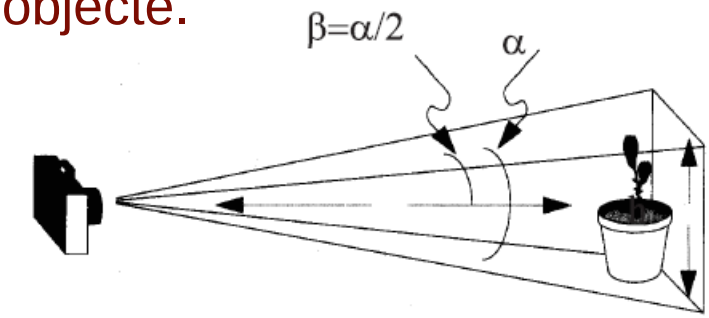
// Definició projecció Perspectiva
glm:mat4 ProjectionMatrix = glm::perspective(radians(60.f),1.0f,0.1f,200.0f);
glm::mat4      ViewMatrix      =      glm:lookAt(glm::vec3(10.0f,0.0f,0.0f),
glm::vec3(0.0f,0.0f,0.0f),glm::vec3(0.0f,0.0f,1.0f)); // Definició càmera.

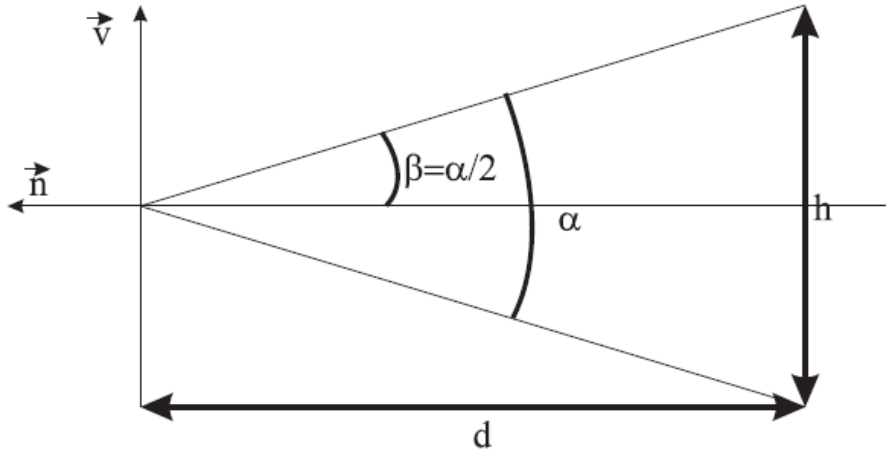
<Activació shader>
<Pas de matrius ProjectionMatrix, ViewMatrix al shader>
Dibuixa_Escena();        // Dibuix de l'escena al buffer OpenGL,
                         //    incloent TG als objectes.
swapBuffers();           // Transferir buffer OpenGL a buffer pantalla.
```

4.2. Projecció

```
glm::mat4 glm::perspective(float fovy, float aspect, float znear,
float zfar);
```

Càlcul del camp de visió òptim: Donat un objecte o escena amb una envolvent d'alçada h i una distància d de la càmera, volem saber l'angle de visió per a poder veure tot l'objecte.





$$\tan\beta = \frac{\frac{h}{2}}{d} = \frac{h}{2d}$$

$$\beta = \arctan\frac{h}{2d}$$

$$\alpha = 2\arctan\frac{h}{2d}$$

3. Transformacions Geomètriques en GLM

Projecció arbitrària: Caballera

Matriu:

$$(x_p, y_p, z_p, 1) = \begin{pmatrix} 1 & 0 & L_1 \cos\phi & 0 \\ 0 & 1 & L_1 \sin\phi & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_v \\ y_v \\ z_v \\ 1 \end{pmatrix}$$

Implementació:

```
glm::mat4 M(1.0f);
…
glm::mat4 Mc(1.0f);                                  // Msh ← I
Mc[0] = vec4(1.0f,0.0f,0.0f,0.0f);                   // Primera columna
Mc[1] = vec4(0.0f,1.0f,0.0f,0.0f);                   // Segona columna
Mc[2] = vec4(L1*cos(ϕ),L1*sin(ϕ),0.0f,0.0f);         // Tercera columna
Mc[3] = vec4(0.0f,0.0f,0.0f,1.0f);                   // Quarta columna
M = M * Mc;
<Activació shader>
<Pas de matriu M al shader>
Dibuixa_Escena();  // Dibuix de l'escena al buffer OpenGL,
                   // Càlcul v' = M * v al shader (v vèrtex escena)
swapBuffers();     // Transferir buffer OpenGL a buffer pantalla.
```

4.3. Viewport

Definició en píxels de l'àrea en pantalla on volem visualitzar l'escena.

Origen (0,0) en l'extrem esquerra inferior.

Matriu de *Windowing* 2D, darrer pas per a obtenir els vèrtexs en coordenades pantalla.

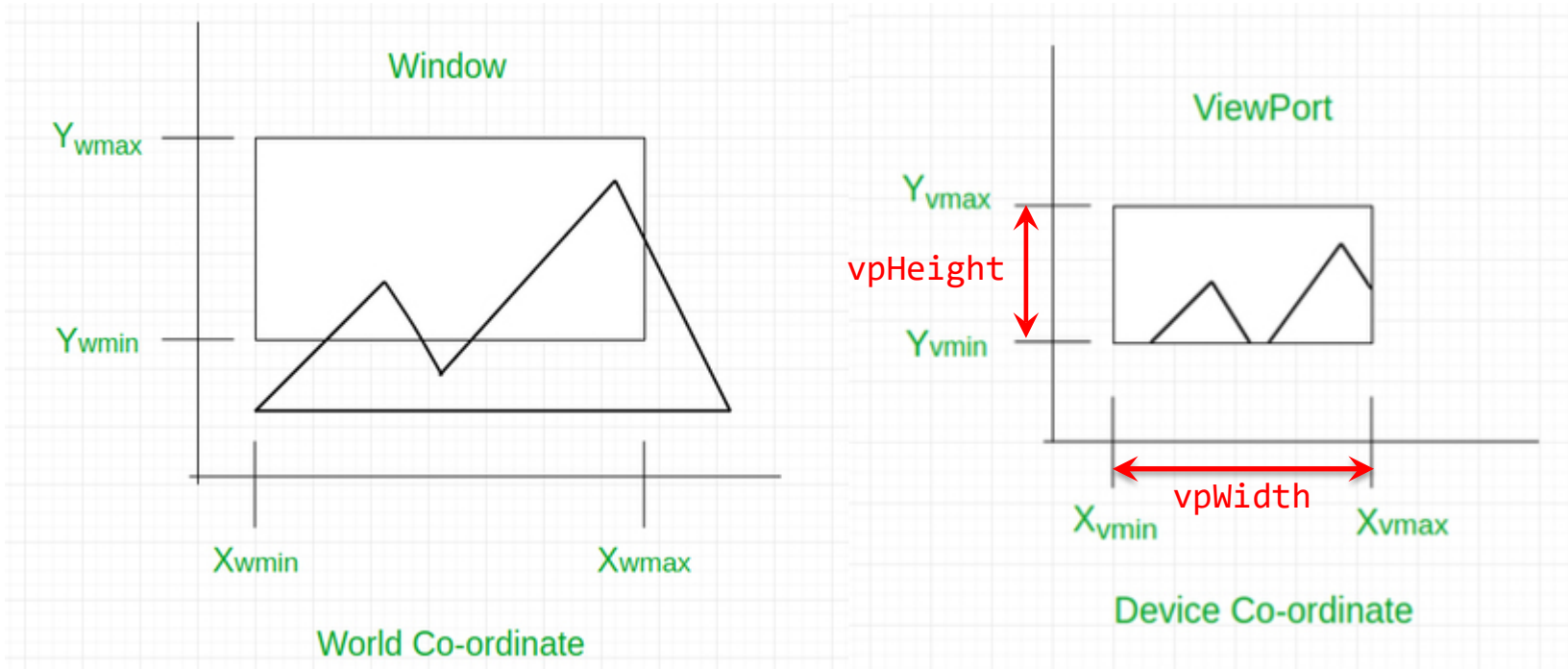
Una única matriu per a definir aquesta transformació (no acumulació de matrius).

Comanda `glViewport()`.

4.3. Viewport

```
void glViewport(GLint Xvmin, GLint Yvmin, GLsizei vpWidth, GLsizei vpHeight);
```

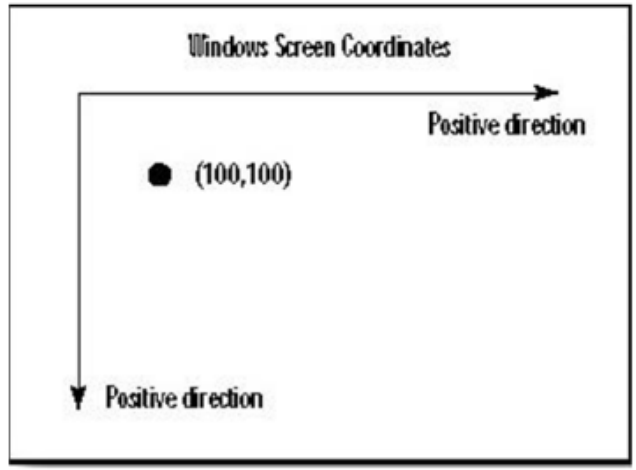
Acció: Defineix la matriu de transformació de windowing 2D amb la posició i el tamany del viewport com un quadrilàter amb l'extrem inferior esquerra a `(Xvmin,Yvmin)` amb amplada `vpWidth` i alçada `vpHeight`.



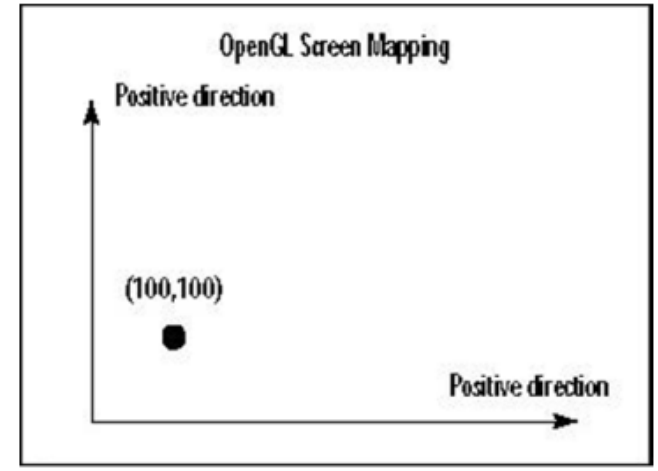
4.3. Viewport

Viewport definit sobre buffer de dibuix OpenGL amb l'origen (0,0) en la part inferior esquerra (*OpenGL Screen Mapping*).

Drivers placa gràfica realitzen darrera transformació sobre el buffer de pantalla, usualment amb l'origen (0,0) en la part superior esquerra (*Window Screen Coordinates*) quan es traspassa el buffer OpenGL al buffer de la placa gràfica (`swapbuffers()`).



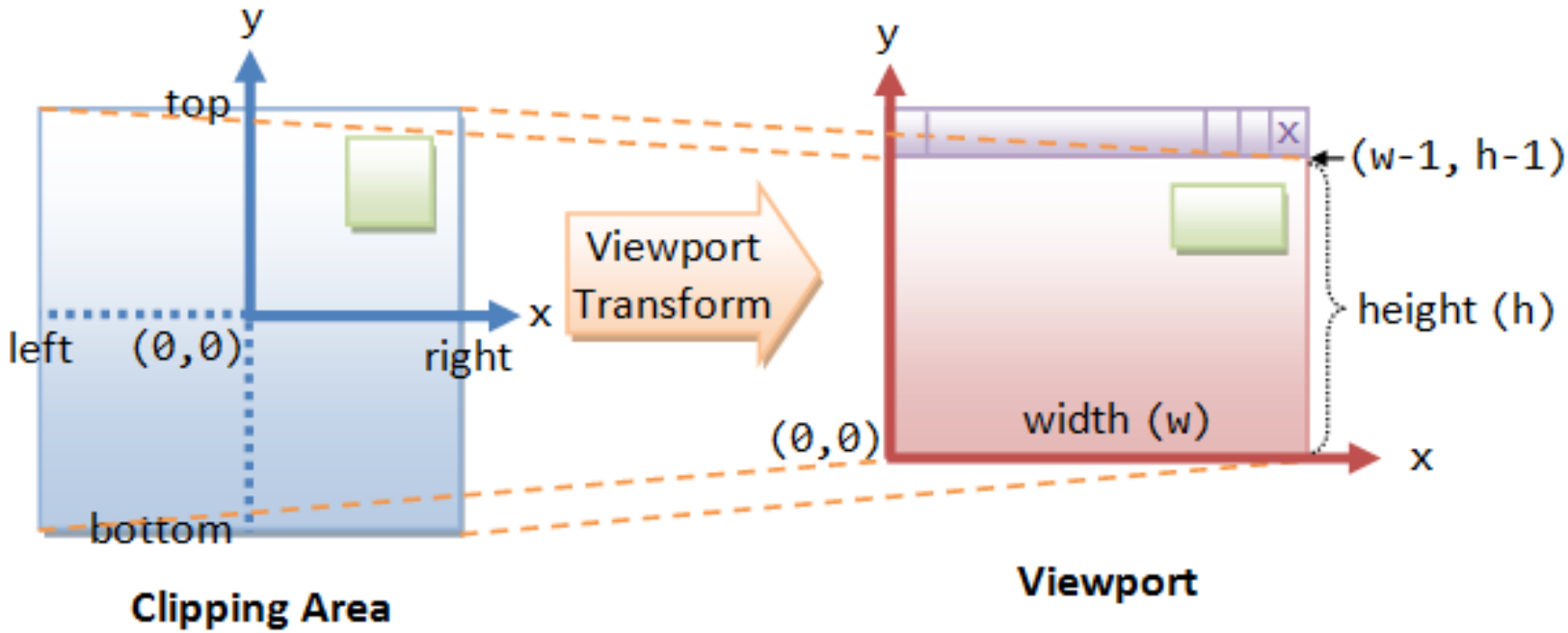
Default Windows screen coordinate mapping



Default OpenGL window coordinate mapping

4.3. Viewport

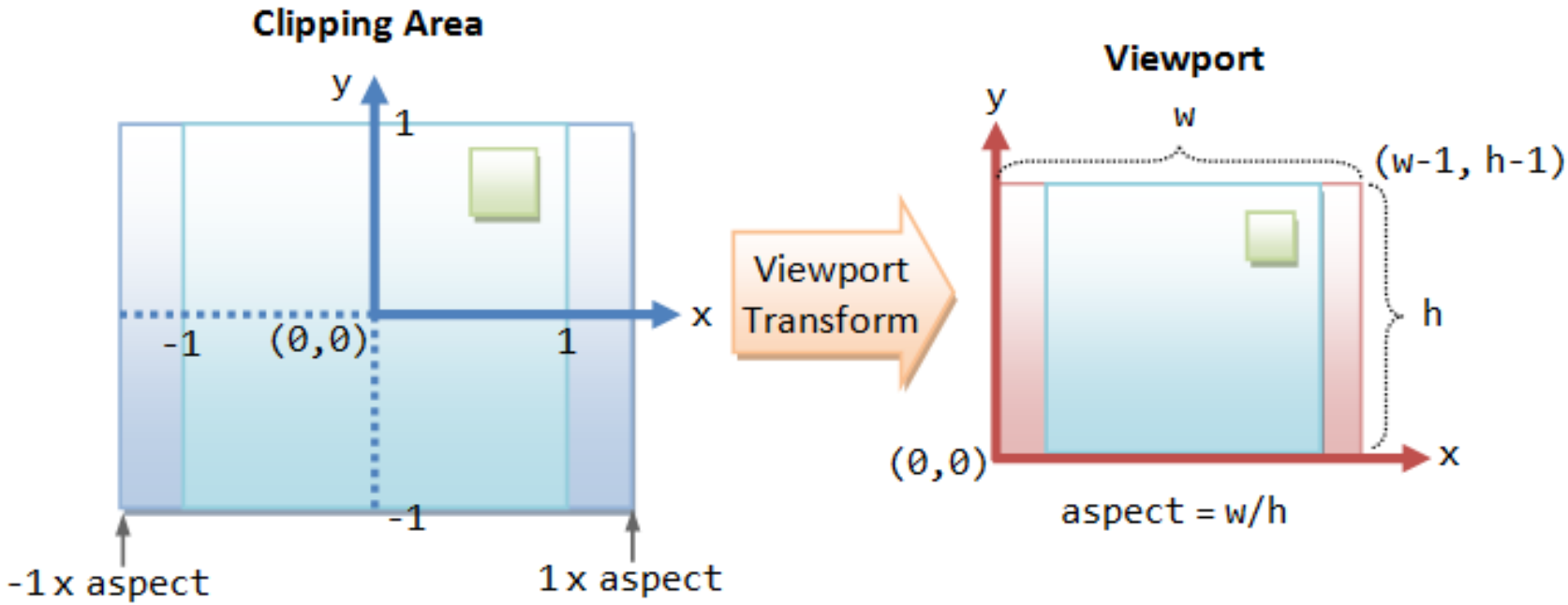
Transformació Window-Viewport pot dibuixar els objectes a l'escena desproporcionats pels diferents aspect ratios entre el clipping area i el viewport.



Clipping Area and Viewport: Objects will be distorted if the aspect ratios of the clipping area and viewport are different.

4.3. Viewport

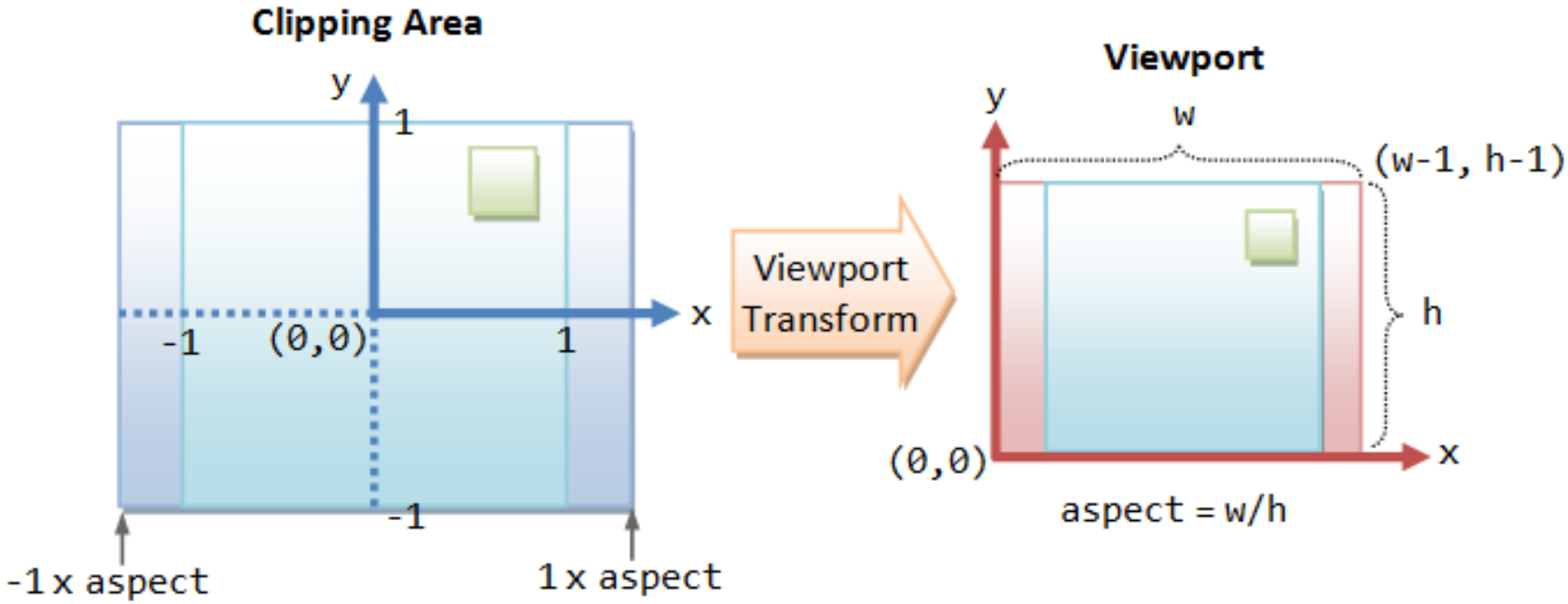
Per a poder veure els objectes proporcionats, cal que l'aspect ratio del Clipping Area (depenent del Volum de Visualització) i el del Viewport siguin iguals.



Clipping Area and Viewport: same aspect ratio for the clipping area and viewport to ensure that the objects are not distorted.

4.3. Viewport

Donat que l'aspect ratio és fixat per la finestra, cal adequar les coordenades de la Clipping Area (definit del Volum de Visualització) per a que tingui el mateix aspect ratio que el Viewport.



Clipping Area and Viewport: same aspect ratio for the clipping area and viewport to ensure that the objects are not distorted.

4.3. Viewport

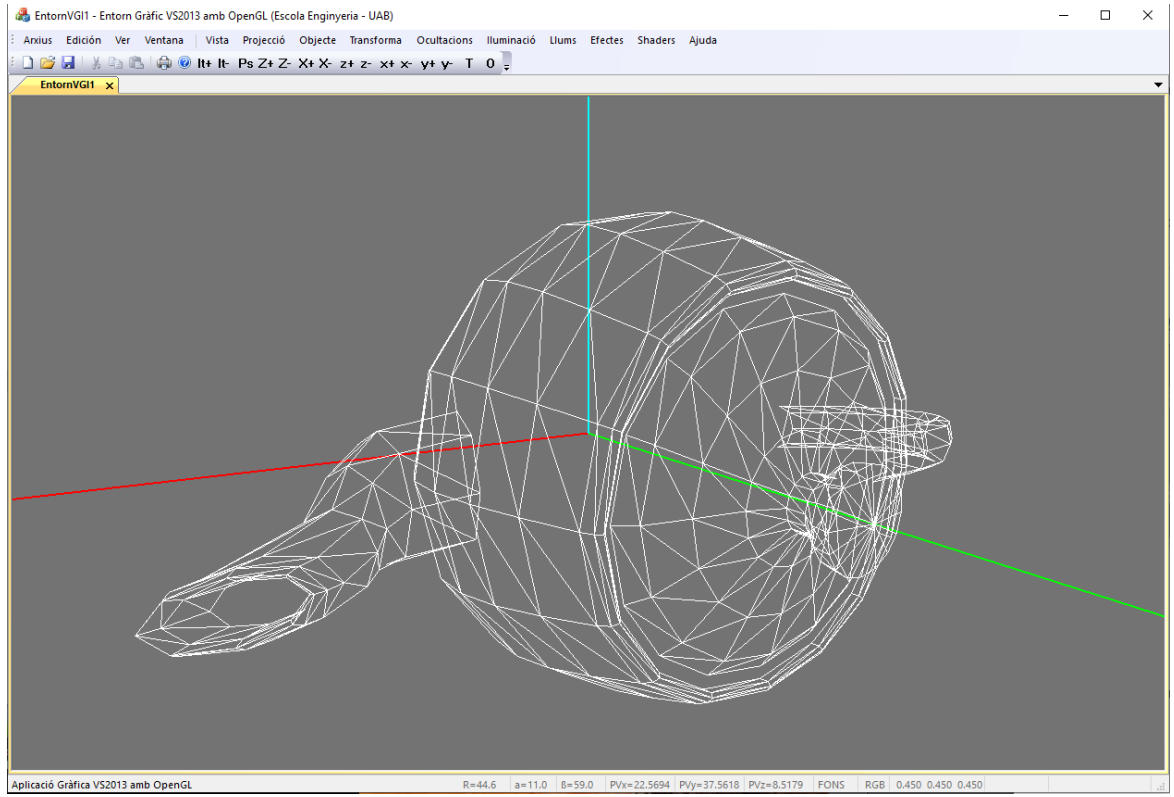
Dos tipus de presentació en pantalla:

1. **MonoViewport.** Un únic Viewport que ocupi o no tota la finestra.
2. **Multiviewport.** Definir vàries vistes i cada una en un diferent viewport.

4.3.1. MonoViewport

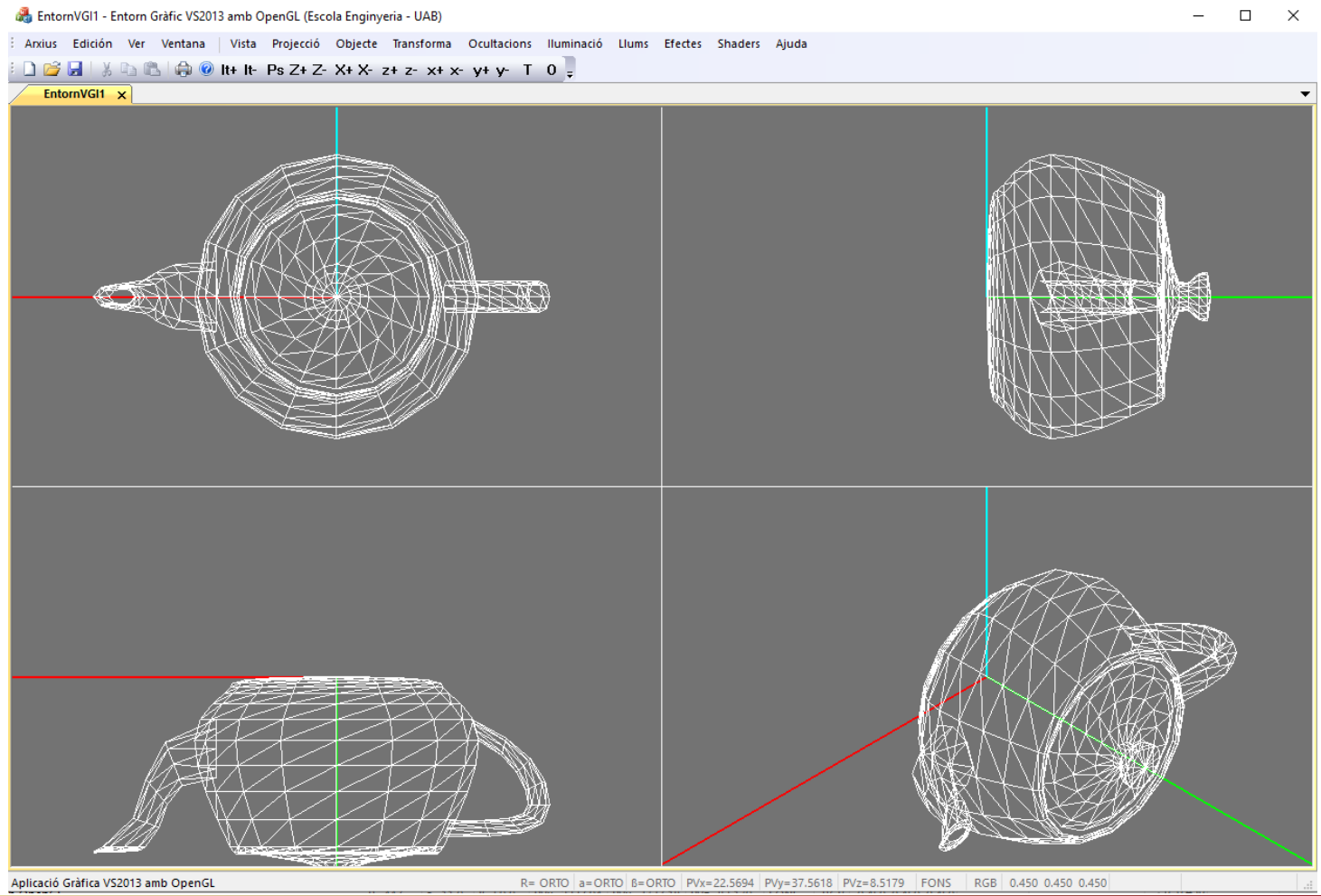
Definir un únic viewport que agafi tota l'amplada i alçada de la finestra:

```
glViewport(0,0,w,h);    // w,h amplada i alçada de la finestra
```



4.3.1. MultiViewport

Definir vàries vistes, cadascuna en un viewport diferent.



4.3.1. MultiViewport

En cada vista s'esborra pantalla.

Per a garantir que es preservi la vista anterior quan es dibuixa una nova, cal retallar el Viewport.

Retall de Viewport: Es defineix en el *Scissor Buffer* d'OpenGL la zona de pantalla (el nostre viewport) on volem que es dibuixi.

Comandes de retall de Viewport:

- `glScissor()` : Definició de l'àrea de retall.
- `glEnable(GL_SCISSOR_TEST)`: Activar el retall de Viewport.
- `glDisable(GL_SCISSOR_TEST)`: Desactivar el retall de Viewport.

4.3.1. MultiViewport

```
void glScissor(GLint x, GLint y, GLsizei width, GLsizei height);
```

Acció: Defineix la finestra de retall (*scissor box*) quan s'activa el retall de viewport. Els paràmetres referencien una finestra amb el mateix sistema de coordenades que el definit `glViewport()`.

Exemple:

```
glEnable(GL_SCISSOR_TEST);        // Activar retall de Viewport
glViewport(0,0,w/2,h/2);          // Definir Viewport Vista 1
glScissor(0,0,w/2,h/2);           // Definir Scissor Box
glm::mat4 ProjectionMatrix = glm::perspective(radians(60.0f),
                                              1.0f,0.1f,200.0f);
        …
glm::mat4      ViewMatrix      =      glm:lookAt(glm::vec3(10.0f,0.0f,0.0f),
glm::vec3(0.0f,0.0f,0.0f),glm::vec3(0.0f,0.0f,1.0f)); // Definició càmera.
Dibuixa_Escena();
        …
glViewport(w/2,0,w/2,h/2);        // Definir Viewport Vista 2
glScissor(w/2,0,w/2,h/2);         // Definir Scissor Box
        …
glDisable(GL_SCISSOR_TEST);       // Desactivar Retall de Viewport
```

4. Transformacions de Visualització

Exemple de codi Visualització d'una Vista:

```
glViewport(0,0,w,h);     // Definir Viewport de la Vista
                         //     (tota la pantalla)

// Definició projecció Ortogràfica | Perspectiva
glm:mat4 ProjectionMatrix = glm::ortho(-10.0f,10.0f,-10.0f,10.0f,0.1f,200.0f);
|
glm::mat4 ProjectionMatrix = glm::perspective(radians(60.0f), 1.0f,0.1f,200.0f);
glm::mat4        ViewMatrix        =        glm:lookAt(glm::vec3(10.0f,0.0f,0.0f),
glm::vec3(0.0f,0.0f,0.0f),glm::vec3(0.0f,0.0f,1.0f)); // Definició càmera.

<Activació shader>
<Pas de matrius ProjectionMatrix, ViewMatrix al shader>
Dibuixa_Escena();        // Dibuix de l'escena al buffer OpenGL,
                         // incloent TG als objectes.
swapBuffers();           // Transferir buffer OpenGL a buffer pantalla.
```

5. Pas de Matrius GLM als shaders

Un cop calculades les matrius en l'aplicació, cal passar-les al shader per a que s'apliquin en el pipeline de visualització per a cada vèrtex de l'escena.

En l'aplicació usualment es defineixen 3 matrius que contenen:

- `projectionMatrix`: Transformació de projecció.
- `viewMatrix`: Transformació de càmera.
- `modelMatrix`: Transformacions geomètriques per a l'objecte de l'escena.

Per a estalviar càlculs als shaders es poden calcular des de l'aplicació:

```
ModelViewMatrix = viewMatrix * modelMatrix;
ModelViewProjectionMatrix = projectionMatrix * viewMatrix * modelMatrix;
```

5. Pas de Matrius GLM als shaders

El tipus `glm::mat4` és compatible amb el tipus `mat4` de GLSL.

Matrius passades com uniform als shaders.

En l'aplicació:

```
glm::mat4 projectionMatrix(1.0f), viewMatrix(1.0f), modelMatrix(1.0f);
glm::mat4  ModelViewProjectionMatrix=  projectionMatrix  *  viewMatrix  *
modelMatrix;
…
// Activació shader
glUseProgram(shaderId);
// Pas de la matrius ModelViewProjectionMatrix al shader:
glUniformMatrix4fv(glGetUniformLocation(shaderId,  'transform'),1,GL_FALSE,
glm::value_ptr(ModelViewProjectionMatrix));

Dibuixa_Objecte();       // Dibuix de l'objecte de l'escena al buffer OpenGL
swapBuffers();           // Transferir buffer OpenGL a buffer pantalla.
```

5. Pas de Matrius GLM als shaders

En Vertex Shader:

```
#version 330 core
layout (location=0) in vec3 in_Vertex;
layout (location=1) in vec3 in_Color;
layout (location=2) in vec3 in_TexCoord;

out vec3 outColor;
out vec3 texCoord;

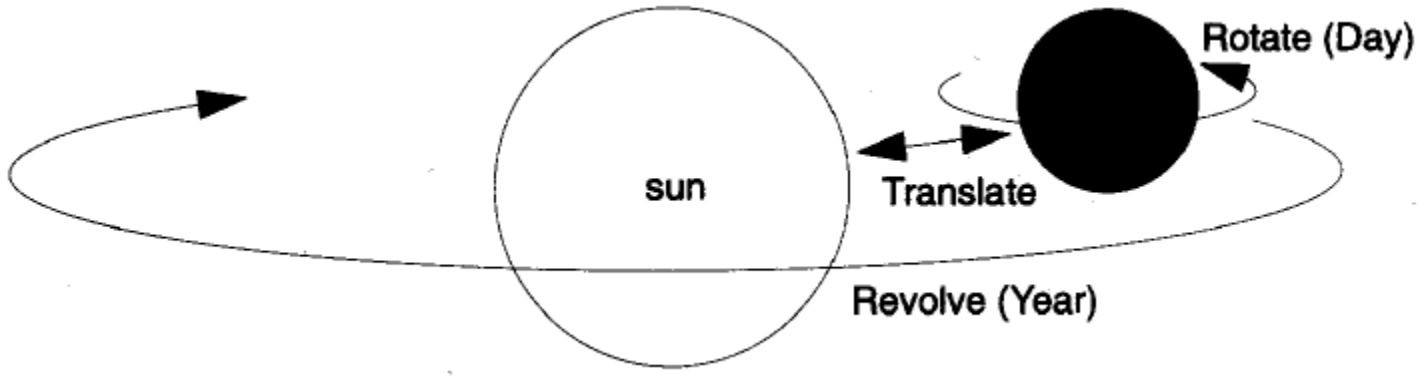
uniform mat4 transform;

void main()
{
  gl_Position = transform * vec(in_vertex,1.0f);
  outColor = in_Color;
  texCoord = vec2(in_texCoord.x,1.0-in_texCoord.y);
}
```

5. Exemple

Representar el moviment del Sol, la Terra i la Lluna:

- **Sol:** Centrat a l'orígen. Esfera de radi 10.
- **Terra:** Esfera de radi 2 amb dues rotacions:
  - Respecte el seu propi eix (variable `rdia`)
  - Respecte el Sol (variable `rany`) a una distancia de 5 del Sol.
- **Lluna:** Esfera de radi 0.4 rotant segons una variable `rmes` al volt de la Terra a una distancia 1.



5. Exemple

Sol: Esfera de radi 10 centrada a l'orígen.

Codi:

```
// Dibuix del Sol
CColor color_groc;
…
color_groc.r = 0.0; color_groc.g = 1.0;
color_groc.b = 1.0; color_groc.a = 1.0);    // Color groc
SeleccionaColor(sh_programID, color_groc, sw_material);
glSolidSphere(10.0,20,20);   // Dibuix del Sol
```

5. Exemple

Terra: Esfera de radi 2 aplicant les transformacions:

1. Dibuixar esfera de radi 2 centrada a l'orígen
2. Rotació de 23 graus respecte l'eix Y.
3. Traslació de (0.0,5.0,0.0)
4. Rotació de `rany` graus respecte l'eix Z (0,0,1).

Codi:

```
// Dibuix de la Terra
glm::mat4 MT(1.0f);
MT = glm::rotate(MT,radians(rany),glm::vec3(0.0f,0.0f,1.0f);   // Pas 4
MT = glm::translate(MT,glm::vec3(0.0f,5.0f,0.0f);              // Pas 3
MT = glm::rotate(MT,radians(23.0f),glm::vec3(0.0f,1.0f,0.0f);  // Pas 2
SeleccionaColor(sh_programID, color_blau, sw_material);        // Color blau
<Activació shader>
<Pas de matriu MT i color al shader>
glSolidSphere(2.0,20,20);                                      // Pas 1
```

5. Exemple

Sol i Terra

Codi:

```
// Dibuix del Sol
…
glSolidSphere(10,20,20); // Dibuix del Sol

// Dibuix de la Terra
glm::mat4 MT(1.0f);
MT = glm::rotate(MT,radians(rany),glm::vec3(0.0f,0.0f,1.0f);      // Pas 4
MT = glm::translate(MT,glm::vec3(0.0f,5.0f,0.0f);                 // Pas 3
MT = glm::rotate(MT,radians(23.0f),glm::vec3(0.0f,1.0f,0.0f);     // Pas 2
SeleccionaColor(sh_programID, color_blau, sw_material);       // Color blau
<Activació shader>
<Pas de matriu MT i color al shader>
glSolidSphere(2.0,20,20);                                         // Pas 1
```

5. Exemple

Lluna: Esfera de radi 0.4 aplicant les transformacions:

1. Dibuixar esfera de radi 0.4 centrada a l'orígen
2. Traslació de (0,1,0)
3. Rotació de `rdia` graus respecte l'eix Z (0,0,1).
4. Transformacions que s'apliquen a la Terra per a col·locar la Lluna al volt de la Terra.
   a) Traslació de (0.0,5.0,0.0)
   b) Rotació de `rany` graus respecte l'eix Z (0,0,1).

Codi:

```
glm::mat4 ML(1.0f);                                          // Dibuix de la Lluna
ML = glm::rotate(ML,radians(rany),glm::vec3(0.0f,0.0f,1.0f);  // Pas 4b
ML = glm::translate(ML,glm::vec3(0.0f,5.0f,0.0f);             // Pas 4a
ML = glm::rotate(ML,radians(rdia),glm::vec3(0.0f,0.0f,1.0f);  // Pas 3
ML = glm::translate(ML,glm::vec3(0.0f,1.0f,0.0f);             // Pas 2
SeleccionaColor(sh_programID, color_blanc, sw_material);      // Color blanc
<Activació shader>
<Pas de matriu ML i color al shader>
gluSphere(0.4,20,20);                                        // Pas 1
```

5. Exemple

```
void STL(GLfloat rdia, GLfloat rany);
{  // Dibuix del Sol …
   glSolidSphere(10,20,20); // Dibuix del Sol
// Dibuix de la Terra
   glm::mat4 MT(1.0f);
   MT = glm::rotate(MT,radians(rany),glm::vec3(0.0f,0.0f,1.0f);   // Pas 4
   MT = glm::translate(MT,glm::vec3(0.0f,5.0f,0.0f);              // Pas 3
   MT = glm::rotate(MT,radians(23.0f),glm::vec3(0.0f,1.0f,0.0f); // Pas 2
   SeleccionaColor(sh_programID, color_blau, sw_material);        // Color blau
   <Activació shader>
   <Pas de matriu MT i color al shader>
   gluSphere(2.0,20,20);                                          // Pas 1
// Dibuix de la Lluna
   glm::mat4 ML(1.0f);
   ML = glm::rotate(ML,radians(rany),glm::vec3(0.0f,0.0f,1.0f);   // Pas 4b
   ML = glm::translate(ML,glm::vec3(0.0f,5.0f,0.0f);              // Pas 4a
   ML = glm::rotate(ML,radians(rdia),glm::vec3(0.0f,0.0f,1.0f);   // Pas 3
   ML = glm::translate(ML,glm::vec3(0.0f,1.0f,0.0f);              // Pas 2
   ML = glm::rotate(ML,radians(23.0f),glm::vec3(0.0f,1.0f,0.0f); // Pas 2
   SeleccionaColor(sh_programID, color_blanc, sw_material);       // Color blanc
   <Activació shader>
   <Pas de matriu ML i color al shader>
   gluSphere(0.4,20,20);                                          // Pas 1
}
```

5. Exemple

```
void STL(GLfloat rdia, GLfloat rany);
{  // Dibuix del Sol
   …
   glSolidSphere(10,20,20); // Dibuix del Sol

// Dibuix de la Terra
   glm::mat4 MT(1.0f), ML(1.0f);
   ML = glm::rotate(ML,radians(rany),glm::vec3(0.0f,0.0f,1.0f);   // Pas 4, Pas 4b
   ML = glm::translate(ML,glm::vec3(0.0f,5.0f,0.0f);              // Pas 3, Pas 4a
   MT = glm::rotate(ML,radians(23.0f),glm::vec3(0.0f,1.0f,0.0f); // Pas 2
    SeleccionaColor(sh_programID, color_blau, sw_material);      // Color blau
   <Activació shader>
   <Pas de matriu MT i color al shader>
   gluSphere(2.0,20,20);                                         // Pas 1

// Dibuix de la Lluna
   ML = glm::rotate(ML,radians(rdia),glm::vec3(0.0f,0.0f,1.0f);   // Pas 3
   ML = glm::translate(ML,glm::vec3(0.0f,1.0f,0.0f);              // Pas 2
   ML = glm::rotate(MT,radians(23.0f),glm::vec3(0.0f,1.0f,0.0f); // Pas 2
    SeleccionaColor(sh_programID, color_blau, sw_material);      // Color blanc
   <Activació shader>
   <Pas de matriu ML i color al shader>
   gluSphere(0.4,20,20);                                         // Pas 1
}
```

En resum...

- Per implementar transformacions geomètriques i de visualització necessitem definir matrius 4x4.
- En GL 3.3+ les estructures de piles de matrius (`GL_MODELVIEW`, `GL_PROJECTION`), les comandes de transformacions geomètriques (`glTranslate*()`, `glScale*()`, `glRotate*()`), la de càmera (`gluLookAt()`) i les de projecció (`glOrtho()`, `glFrustrum()`, `gluPerspective()`) estan deprecades.
- Cal utilitzar una llibreria externa com GLM per implementar-les en l'aplicació i passar-les com a variables `uniform` en els shaders per a implementar el pipeline de visualització.